

AIM: A. Create and deploy a block chain network using Hyperledger Fabric

B. Create and deploy a block chain network using Hyperledger Fabric(With user input)

1. AssertTransfer.js

```
'use strict';

const { Contract } = require('fabric-contract-api');

class AssetTransfer extends Contract {

  // 1. Initialize the ledger with default assets
  async InitLedger(ctx) {
    const assets = [
      { ID: 'asset1', Color: 'blue', Size: 5, Owner: 'Tom', AppraisedValue: 300 },
      { ID: 'asset2', Color: 'red', Size: 10, Owner: 'Brad', AppraisedValue: 400 },
    ];

    for (const asset of assets) {
      asset.docType = 'asset';
      await ctx.stub.putState(asset.ID, Buffer.from(JSON.stringify(asset)));
    }
  }

  // 2. CreateAsset - Issues a new asset after checking if it already exists
  async CreateAsset(ctx, id, color, size, owner, appraisedValue) {
    const exists = await this.AssetExists(ctx, id);
    if (exists) {
      throw new Error(`The asset ${id} already exists`);
    }

    const asset = {
      ID: id,
      Color: color,
      Size: parseInt(size),
      Owner: owner,
      AppraisedValue: parseInt(appraisedValue),
      docType: 'asset',
    };

    await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
    return JSON.stringify(asset);
  }

  // 3. ReadAsset - Returns the asset stored in the world state
  async ReadAsset(ctx, id) {
    const assetJSON = await ctx.stub.getState(id);
    if (!assetJSON || assetJSON.length === 0) {
      throw new Error(`The asset ${id} does not exist`);
    }
    return assetJSON.toString();
  }
}
```

```
// 4. UpdateAsset - Updates details of an existing asset
async UpdateAsset(ctx, id, color, size, owner, appraisedValue) {
  const exists = await this.AssetExists(ctx, id);
  if (!exists) {
    throw new Error(`The asset ${id} does not exist`);
  }
  // Overwriting original asset details
  const updatedAsset = {
    ID: id,
    Color: color,
    Size: parseInt(size),
    Owner: owner,
    AppraisedValue: parseInt(appraisedValue),
    docType: 'asset',
  };
  await ctx.stub.putState(id, Buffer.from(JSON.stringify(updatedAsset)));
  return JSON.stringify(updatedAsset);
}

// 5. TransferAsset - Updates only the owner field of an asset
async TransferAsset(ctx, id, newOwner) {
  const assetString = await this.ReadAsset(ctx, id);
  const asset = JSON.parse(assetString);
  asset.Owner = newOwner;
  await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
  return JSON.stringify(asset);
}

// 6. DeleteAsset - Removes an asset completely from the world state
async DeleteAsset(ctx, id) {
  const exists = await this.AssetExists(ctx, id);
  if (!exists) {
    throw new Error(`The asset ${id} does not exist`);
  }
  return await ctx.stub.deleteState(id);
}

// 7. AssetExists - Helper utility returns true when asset with given ID exists
async AssetExists(ctx, id) {
  const assetJSON = await ctx.stub.getState(id);
  return assetJSON && assetJSON.length > 0;
}

// 8. GetAllAssets - Retrieves all assets currently stored in the world state
async GetAllAssets(ctx) {
  const allResults = [];
  // range query with empty string for startKey and endKey does an open-ended
  resume of all keys
  const iterator = await ctx.stub.getStateByRange("", "");
  let result = await iterator.next();

  while (!result.done) {
    const strValue = Buffer.from(result.value.value.toString()).toString('utf8');
```

```

    let record;
    try {
      record = JSON.parse(strValue);
    } catch (err) {
      console.log(err);
      record = strValue;
    }
    allResults.push(record);
    result = await iterator.next();
  }
  return JSON.stringify(allResults);
}
}
module.exports = AssetTransfer;

```

A. Create and deploy a block chain network using Hyperledger Fabric(without user input)

1. Test.js code.

```

const AssetTransfer = require('./AssetTransfer');

// 1. Simulate Hyperledger Fabric's World State database in-memory
const mockState = {};

const mockCtx = {
  stub: {
    // Simulates saving bytes to a blockchain state database
    putState: async (key, value) => {
      mockState[key] = value.toString();
    },
    // Simulates fetching bytes from the database
    getState: async (key) => {
      return mockState[key] ? Buffer.from(mockState[key]) : Buffer.alloc(0);
    },
    // Simulates removing a key from active state
    deleteState: async (key) => {
      delete mockState[key];
    },
    // Simulates the GetAllAssets range scanner
    getStateByRange: async (startKey, endKey) => {
      const keys = Object.keys(mockState).sort();
      let index = 0;
      return {
        next: async () => {
          if (index < keys.length) {
            const currentKey = keys[index++];
            return {
              value: {
                key: currentKey,
                value: Buffer.from(mockState[currentKey])
              },
            },
          }
        },
      };
    },
  },
};

```

```
        done: false
      };
    }
    return { done: true };
  }
};
}
```

```
// 2. Run your transactions sequentially
async function testBlockchainLifecyle() {
  const contract = new AssetTransfer();

  console.log("=== 1. Initializing Ledger with default records ===");
  await contract.InitLedger(mockCtx);
  console.log("Initialization Complete.\n");

  console.log("=== 2. Reading 'asset1' ===");
  const asset1 = await contract.ReadAsset(mockCtx, 'asset1');
  console.log("Result:", asset1, "\n");

  console.log("=== 3. Creating a new asset ('asset3') ===");
  await contract.CreateAsset(mockCtx, 'asset3', 'green', '15', 'Alice', '600');
  console.log("Asset 3 successfully registered.\n");

  console.log("=== 4. Attempting to create duplicate 'asset3' (Error test) ===");
  try {
    await contract.CreateAsset(mockCtx, 'asset3', 'yellow', '5', 'Bob', '100');
  } catch (error) {
    console.log("Caught expected safety block:", error.message, "\n");
  }

  console.log("=== 5. Transferring ownership of 'asset2' to 'Charlie' ===");
  const transferred = await contract.TransferAsset(mockCtx, 'asset2', 'Charlie');
  console.log("Updated Asset:", transferred, "\n");

  console.log("=== 6. Fetching ALL active assets on our simulated ledger ===");
  const allAssets = await contract.GetAllAssets(mockCtx);
  console.log(JSON.parse(allAssets));
}
testBlockchainLifecyle().catch(console.error);
```

OUTPUT:**A.Initialize the node npm package.**

```
C:\fabric-contract>npm init -y
Wrote to C:\fabric-contract\package.json:

{
  "name": "fabric-contract",
  "version": "1.0.0",
  "description": "",
  "main": "AssetTransfer.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

B.Install the fabric mock-up.

```
C:\fabric-contract>npm install fabric-contract-api

added 35 packages, and audited 36 packages in 5s

2 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

C.Run the test.js file.

```
C:\fabric-contract>node test.js
=== 1. Initializing Ledger with default records ===
Initialization Complete.

=== 2. Reading 'asset1' ===
Result: {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tom","AppraisedValue":300,"docType":"asset"}

=== 3. Creating a new asset ('asset3') ===
Asset 3 successfully registered.

=== 4. Attempting to create duplicate 'asset3' (Error test) ===
Caught expected safety block: The asset asset3 already exists

=== 5. Transferring ownership of 'asset2' to 'Charlie' ===
Updated Asset: {"ID":"asset2","Color":"red","Size":10,"Owner":"Charlie","AppraisedValue":400,"docType":"asset"}

=== 6. Fetching ALL active assets on our simulated ledger ===
[
  {
    ID: 'asset1',
    Color: 'blue',
    Size: 5,
    Owner: 'Tom',
    AppraisedValue: 300,
    docType: 'asset'
  },
  {
    ID: 'asset2',
    Color: 'red',
    Size: 10,
    Owner: 'Charlie',
    AppraisedValue: 400,
    docType: 'asset'
  },
  {
    ID: 'asset3',
    Color: 'green',
    Size: 15,
    Owner: 'Alice',
    AppraisedValue: 600,
    docType: 'asset'
  }
]
```

B. Create and deploy a block chain network using Hyperledger Fabric(With user input).**1. Input.js**

```
const readline = require('readline');
const AssetTransfer = require('./AssetTransfer');

// 1. In-memory storage acting as our blockchain database
const mockState = {};
const mockCtx = {
  stub: {
    putState: async (key, value) => { mockState[key] = value.toString(); },
    getState: async (key) => mockState[key] ? Buffer.from(mockState[key]) :
Buffer.alloc(0),
    deleteState: async (key) => { delete mockState[key]; },
    getStateByRange: async () => {
      const keys = Object.keys(mockState).sort();
      let index = 0;
      return {
        next: async () => {
          if (index < keys.length) {
            const currentKey = keys[index++];
            return { value: { value: Buffer.from(mockState[currentKey]) }, done: false
};
          }
          return { done: true };
        }
      };
    }
  }
};

// Setup command-line prompt interface
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

const askQuestion = (query) => new Promise((resolve) => rl.question(query, resolve));
const contract = new AssetTransfer();

async function main() {
  // Populate with default ledger data first
  await contract.InitLedger(mockCtx);
  console.log("=====");
  console.log(" Fabric Smart Contract CLI Simulator ");
  console.log("=====\\n");

  while (true) {
```

```
console.log("Choose an operation:");
console.log("1. Create New Asset");
console.log("2. Read / View Asset");
console.log("3. Transfer Asset Owner");
console.log("4. View All Assets");
console.log("5. Exit");

const choice = await askQuestion("\nEnter option number (1-5): ");
console.log("\n-----");

try {
  if (choice === '1') {
    const id = await askQuestion("Enter Asset ID (e.g. asset3): ");
    const color = await askQuestion("Enter Color: ");
    const size = await askQuestion("Enter Size (number): ");
    const owner = await askQuestion("Enter Owner Name: ");
    const value = await askQuestion("Enter Appraised Value (number): ");

    console.log("\n[Processing Blockchain Transaction...]");
    const result = await contract.CreateAsset(mockCtx, id, color, size, owner,
value);
    console.log("Success! Created Asset Data:", JSON.parse(result));

  } else if (choice === '2') {
    const id = await askQuestion("Enter Asset ID to lookup: ");

    console.log("\n[Querying Ledger...]");
    const result = await contract.ReadAsset(mockCtx, id);
    console.log("Asset Found:", JSON.parse(result));

  } else if (choice === '3') {
    const id = await askQuestion("Enter Target Asset ID: ");
    const newOwner = await askQuestion("Enter New Owner Name: ");

    console.log("\n[Executing State Transfer...]");
    const result = await contract.TransferAsset(mockCtx, id, newOwner);
    console.log("Transfer Confirmed:", JSON.parse(result));

  } else if (choice === '4') {
    console.log("[Scanning World State DB...]");
    const result = await contract.GetAllAssets(mockCtx);
    console.log("Current Ledger Inventory:\n", JSON.parse(result));

  } else if (choice === '5') {
    console.log("Exiting Simulator. Goodbye!");
    rl.close();
    break;
  }
}
```

```
    } else {
        console.log(" ⚠ Invalid option. Please select 1 through 5.");
    }
} catch (error) {
    // This safely catches smart contract rules (like duplicate IDs or missing assets)
    console.log(" 🚫 Transaction Rejected:", error.message);
}
console.log("-----\n");
}
}
main().catch(console.error);
```

OUTPUT:**1. Create a new asset.**

```
Choose an operation:
1. Create New Asset
2. Read / View Asset
3. Transfer Asset Owner
4. View All Assets
5. Exit

Enter option number (1-5): 1
-----

Enter Asset ID (e.g. asset3): asset101
Enter Color: silver
Enter Size (number): 14
Enter Owner Name: Ethan
Enter Appraised Value (number): 18000

[Processing Blockchain Transaction...]
Success! Created Asset Data: {
  ID: 'asset101',
  Color: 'silver',
  Size: 14,
  Owner: 'Ethan',
  AppraisedValue: 18000,
  docType: 'asset'
}
```

2. Find the asset with its id

```
Enter option number (1-5): 2
```

```
Enter Asset ID to lookup: asset101

[Querying Ledger...]-----
Asset Found: {
  ID: 'asset101',
  Color: 'silver',
  Size: 14,
  Owner: 'Ethan',
  AppraisedValue: 18000,
  docType: 'asset'
}
```


3. Transferring owner asset

```
Enter option number (1-5): 3

-----

Enter Target Asset ID: asset101
Enter New Owner Name: Luther

[Executing State Transfer...]
Transfer Confirmed: {
  ID: 'asset101',
  Color: 'silver',
  Size: 14,
  Owner: 'Luther',
  AppraisedValue: 18000,
  docType: 'asset'
}
```

4. Viewing all assets.

```
Enter option number (1-5): 4

-----

[Scanning World State DB...]
Current Ledger Inventory:
[
  {
    ID: 'asset1',
    Color: 'blue',
    Size: 5,
    Owner: 'Tom',
    AppraisedValue: 300,
    docType: 'asset'
  },
  {
    ID: 'asset101',
    Color: 'silver',
    Size: 14,
    Owner: 'Luther',
    AppraisedValue: 18000,
    docType: 'asset'
  },
  {
    ID: 'asset2',
    Color: 'red',
    Size: 10,
    Owner: 'Brad',
    AppraisedValue: 400,
    docType: 'asset'
  }
]
```

5. Exit.

```
Enter option number (1-5): 5

-----

Exiting Simulator. Goodbye!
```